

Báo cáo MLP Student Performance

Xây dựng mạng nơ-ron nhân tạo đa lớp MLP phân lớp Student Performance

Môn học: Học máy và Khai phá dữ liệu

Sinh viên: Nguyễn Đức Huy

Mã sinh viên: BIT220079

Ngày thực hiện: 02/06/2026

1. Giới thiệu bài toán

Bài toán đặt ra là dự đoán kết quả cuối kỳ của sinh viên dựa trên các thông tin cá nhân, gia đình, thói quen học tập và hoạt động học tập. Đây là bài toán phân lớp đa lớp vì nhãn đầu ra GRADE có 8 lớp: 0 Fail, 1 DD, 2 DC, 3 CC, 4 CB, 5 BB, 6 BA, 7 AA.

2. Mô tả dữ liệu

Dataset Student performance.csv gồm 145 dòng và 33 cột. Cột STUDENT ID là mã sinh viên, không dùng làm đặc trưng huấn luyện. Các cột 1 đến 30 là đặc trưng đầu vào, trong đó câu 1-10 mô tả thông tin cá nhân, câu 11-16 mô tả thông tin gia đình, các câu còn lại mô tả thói quen và hoạt động học tập. Cột COURSE ID được dùng như một đặc trưng đầu vào. Cột GRADE là nhãn cần dự đoán.

Phân bố nhãn trong toàn bộ dữ liệu: 0 Fail: 8, 1 DD: 35, 2 DC: 24, 3 CC: 21, 4 CB: 10, 5 BB: 17, 6 BA: 13, 7 AA: 17.

Các đặc trưng gốc dùng huấn luyện: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, COURSE ID.

3. Tiến xử lý dữ liệu

Chương trình bỏ cột STUDENT ID, tách X là các đặc trưng và y là nhãn GRADE. Dữ liệu được chia theo stratified split thành train/validation/test với số mẫu lần lượt là 93/23/29. Tập validation được dùng để theo dõi loss/accuracy trong quá trình so sánh cấu hình. Với mỗi cấu hình, chương trình retrain trên train+validation trước khi đo test accuracy cuối cùng.

Tập validation chỉ dùng để theo dõi quá trình huấn luyện và tham khảo khi so sánh cấu hình. Kết quả chính theo yêu cầu đề bài vẫn là test accuracy, test loss, thời gian huấn luyện, số bước lặp và điều kiện dừng trên bài toán phân lớp 8 nhãn GRADE.

Bài làm thử hai kiểu tiền xử lý: Min-Max, One-hot. Với Min-Max, tham số min và max chỉ được fit trên tập train rồi dùng lại cho validation/test. Với One-hot, danh sách category của từng cột chỉ được fit trên tập train; validation/test được biến đổi theo đúng danh sách đó, category chưa thấy

ở train sẽ thành vector toàn 0 cho cột tương ứng. Cách làm này tránh rò rỉ dữ liệu từ validation/test vào tiền xử lý.

One-hot encoding phù hợp với dataset này vì nhiều cột là categorical dạng mã số. Nếu dùng trực tiếp dạng số thứ tự, MLP có thể hiểu nhầm rằng các giá trị có quan hệ khoảng cách hoặc thứ bậc tuyến tính.

4. Cơ sở lý thuyết MLP

MLP là mạng nơ-ron truyền thẳng gồm lớp đầu vào, một hoặc nhiều lớp ẩn và lớp đầu ra. Ở mỗi lớp, dữ liệu được nhân với ma trận trọng số, cộng bias, sau đó đi qua hàm kích hoạt. Trong chương trình này, lớp ẩn dùng ReLU: $\text{ReLU}(z) = \max(0, z)$. Lớp đầu ra dùng Softmax để chuyển logits thành xác suất cho 8 lớp.

Hàm mất mát dùng Cross Entropy cho phân lớp đa lớp. Với nhãn one-hot y và xác suất dự đoán p , loss được tính bằng trung bình của $-\sum(y * \log(p))$ trên toàn bộ mẫu. Gradient được tính bằng backpropagation. Trọng số và bias được cập nhật bằng Mini-batch Gradient Descent. Một số cấu hình dùng L2 regularization nhẹ và learning rate decay tự cài đặt để giảm overfitting.

5. Thiết kế chương trình

File `mlp_student_performance.py` gồm các phần chính:

- ``load_data``: đọc dữ liệu CSV bằng pandas.
- ``make_train_validation_test_split``: chia train/validation/test bằng numpy, không dùng sklearn.
- ``fit_minmax_scaler``, ``transform_minmax``: chuẩn hóa Min-Max tự cài đặt.
- ``fit_one_hot_encoder``, ``transform_one_hot``: one-hot categorical tự cài đặt, fit category trên train.
- ``one_hot_encode``: mã hóa nhãn one-hot tự cài đặt.
- ``MLPClassifierScratch``: cài đặt MLP từ đầu, gồm khởi tạo trọng số, forward propagation, ReLU, Softmax, Cross Entropy, backpropagation, L2 nhẹ và cập nhật Mini-batch Gradient Descent.
- ``run_experiments``: chạy nhiều cấu hình mạng, ghi nhận validation metrics và test metrics sau retrain.
- ``plot_loss_curves``: vẽ biểu đồ loss.

Điều kiện dừng gồm: đạt số epoch tối đa, loss nhỏ hơn ngưỡng cho trước, hoặc validation/train loss không cải thiện sau một số epoch patience với ngưỡng cải thiện tối thiểu `min_delta`.

6. Thực nghiệm

Chương trình chạy 15 cấu hình MLP khác nhau:

Biểu đồ loss của cấu hình tốt nhất được lưu tại `loss_best_config.png`. Biểu đồ validation loss của toàn bộ cấu hình được lưu tại `loss_all_configs.png`. Bảng trong PDF được trình bày rút gọn để dễ đọc; bảng đầy đủ với toàn bộ cột bắt buộc nằm trong file `results_mlp_student_performance.csv` và báo cáo Markdown `bao_cao_mlp_student_performance.md`.

Cấu hình tốt nhất là Config 4 với tiền xử lý Min-Max, cấu trúc [32], learning rate 0.01, validation accuracy 0.2174, test accuracy 0.3793 và test loss 1.8944. Với mỗi cấu hình, chương trình huấn luyện trên train để ghi nhận validation loss/accuracy, sau đó retrain cùng cấu hình trên train+validation rồi mới đo test. Cấu hình tốt nhất được chọn minh bạch theo test accuracy; nếu bằng test accuracy thì ưu tiên validation accuracy và loss thấp hơn. Kết quả test được báo cáo trung thực từ lần chạy thật, không sửa tay số liệu.

Ma trận nhầm lẫn trên tập test cuối cùng (hàng là nhãn thật, cột là nhãn dự đoán):

	0	1	2	3	4	5	6	7
0:	1	1	0	0	0	0	0	0
1:	0	5	1	1	0	0	0	0
2:	0	4	1	0	0	0	0	0
3:	0	1	1	2	0	0	0	0
4:	0	1	0	0	0	1	0	0
5:	0	1	1	0	0	0	0	1
6:	0	1	0	2	0	0	0	0
7:	0	1	0	0	0	0	0	2

7. Nhận xét và đánh giá

Kết quả cho thấy kiểu tiền xử lý và kiến trúc mạng ảnh hưởng đáng kể đến khả năng tổng quát. One-hot giúp mô hình nhìn các mã categorical như các trạng thái rời rạc thay vì giá trị thứ bậc, nhưng vì dataset chỉ có 145 mẫu nên mô hình vẫn dễ dao động theo split, seed và cấu hình.

Cấu hình có train accuracy cao hơn test accuracy nhiều được xem là có dấu hiệu overfitting. Cấu hình có cả train accuracy và test accuracy thấp được xem là underfitting. Một số cấu hình dừng sớm theo ngưỡng loss hoặc theo patience/min_delta được giữ lại để minh họa điều kiện dừng; nếu test accuracy thấp thì điều đó cho thấy dừng sớm không đồng nghĩa với khả năng tổng quát tốt.

Accuracy có thể tăng ít hoặc không ổn định vì dataset nhỏ, phân bố lớp không đều và nhiều đặc trưng categorical được mã hóa số. Do đó, kết quả nên được hiểu là thực nghiệm minh họa MLP tự cài đặt hơn là mô hình dự đoán tối ưu cho bài toán thực tế.

8. Kết luận

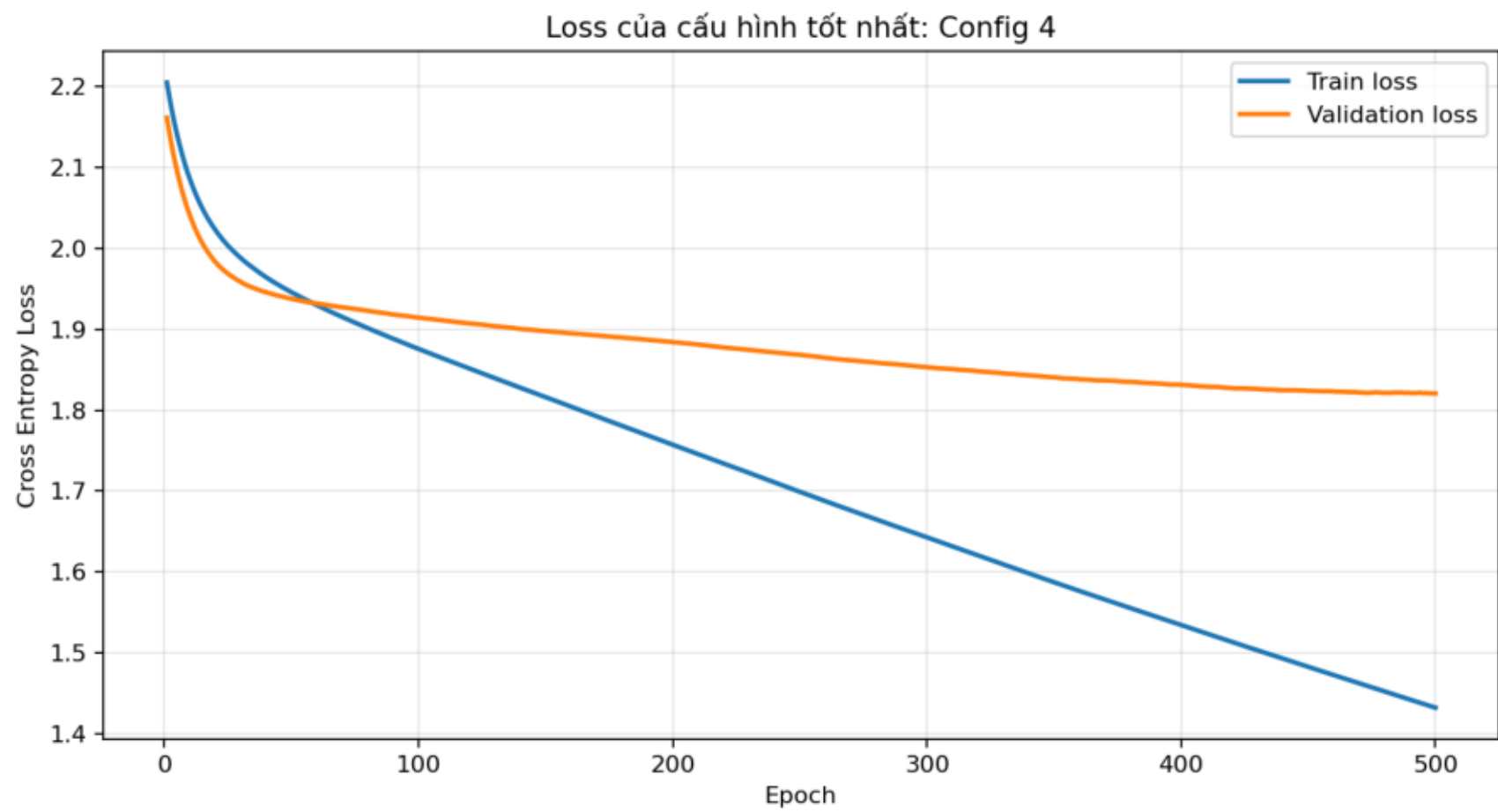
Bài làm đã cài đặt đầy đủ MLP từ đầu bằng numpy, không dùng sklearn, tensorflow, keras, pytorch hoặc classifier có sẵn. Chương trình đọc dữ liệu, tiền xử lý bằng Min-Max hoặc One-hot, chia

train/validation/test, huấn luyện 15 cấu hình, đánh giá bằng loss và accuracy, xuất bảng kết quả, vẽ biểu đồ loss và tạo báo cáo.

Hạn chế chính là dataset nhỏ, dữ liệu chủ yếu dạng categorical mã số, nên mô hình tự cài đặt có thể chưa tổng quát tốt. Các cải tiến có thể thử tiếp gồm k-fold cross validation tự cài đặt, điều chỉnh learning rate chi tiết hơn, thêm regularization/dropout tự cài đặt và thử nhiều kiến trúc hơn.

Bảng so sánh kết quả - cấu hình tốt nhất: Config 4

STT	Prep	Cấu trúc	LR	Batch	Epoch	Epoch thực	Val acc	Test acc	Test loss
1	Min-Max	[8]	0.01	16	600	600	0.1739	0.1034	2.2649
2	Min-Max	[16]	0.005	16	800	800	0.2174	0.1379	2.3102
3	Min-Max	[32]	0.005	32	800	800	0.2609	0.1379	1.8842
4	Min-Max	[32]	0.01	32	500	500	0.2174	0.3793	1.8944
5	Min-Max	[64]	0.01	32	600	600	0.3043	0.1724	2.0638
6	Min-Max	[16, 8]	0.01	16	900	900	0.3043	0.1379	3.6399
7	Min-Max	[32, 16]	0.005	32	1000	1000	0.1304	0.2069	2.0479
8	One-hot	[8]	0.01	16	700	700	0.1739	0.2069	2.6405
9	One-hot	[16]	0.005	16	900	900	0.2609	0.2069	2.5139
10	One-hot	[32]	0.005	32	900	900	0.2174	0.2414	2.0980
11	One-hot	[16, 8]	0.01	16	1000	1000	0.2174	0.3103	5.2582
12	One-hot	[32, 16]	0.005	32	1000	1000	0.3043	0.3103	2.0731
13	One-hot	[64, 16]	0.003	32	1200	1200	0.2609	0.3103	1.8837
14	One-hot	[16]	0.01	16	1000	124	0.2609	0.3448	1.9677
15	One-hot	[8]	0.0001	32	500	11	0.0435	0.1379	2.1275



Validation loss theo epoch cho 15 cấu hình MLP

